DG INTERNS HUB

Week 12 SOC Final Capstone

Threat Detection & Incident Response

Incident ID: DG-SOC-2026-001

Internship Module: SOC Final Capstone

Cybersecurity Internship Programme

Report Date: 10 November 2026

Contents

1. Introduction
2. SOC Architecture
3. Lab Environment and Configuration
4. Wazuh Setup and Configuration
5. Snort IDS Setup and Configuration
6. Incident Scenario
7. Threat Detection
8. Alert Analysis and Investigation
9. Incident Timeline
10. Snort and Wazuh Correlation
11. IOC and Evidence Collection
12. Incident Response Plan
13. Professional SOC Incident Ticket
14. SOC Metrics and Dashboard
15. Challenges Faced and Troubleshooting
16. SOC Analyst Reflection
17. Lessons Learned and Future Improvements
18. Conclusion
Appendix A: Important Terminal Commands
Appendix B: Configuration Snippets
Appendix C: Evidence Descriptions
Appendix D: Additional Investigation Notes

1. Introduction

1.1 What Is a Security Operations Center?

A Security Operations Center (SOC) is a centralised function of people, processes and technology responsible for continuously monitoring an organisation's networks, endpoints, servers and applications, detecting malicious or anomalous activity, and coordinating the response. A SOC provides round-the-clock visibility by collecting telemetry from sensors such as intrusion detection systems (IDS), host agents and security information and event management (SIEM) platforms, and by turning that telemetry into prioritised, actionable alerts.

SOC work is usually organised in tiers. A Tier 1 (L1) analyst monitors the alert queue, validates alerts, enriches them with context and escalates genuine incidents. A Tier 2 analyst performs deeper investigation and containment, and Tier 3 specialists handle threat hunting, forensics and detection engineering. This capstone is written from the perspective of an L1 analyst who detects, triages, documents and recommends response actions for an incident.

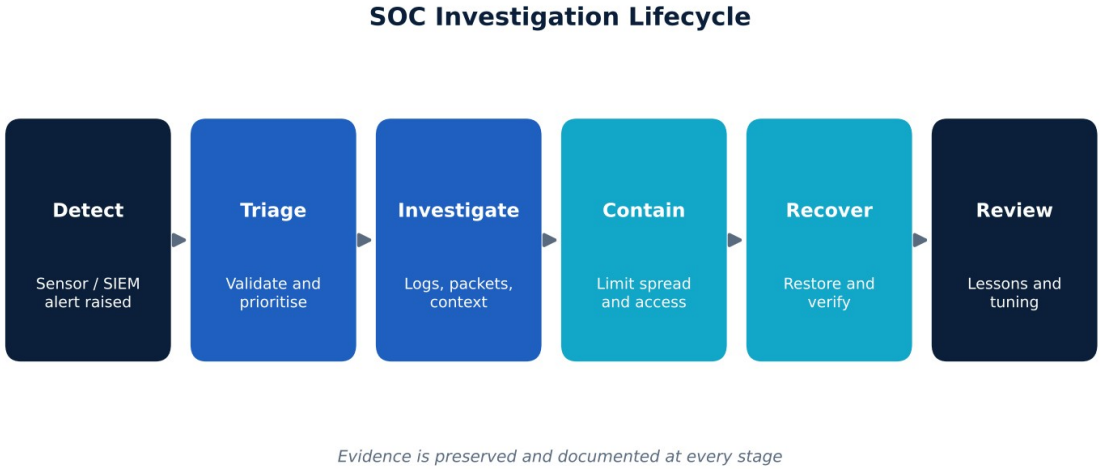


Figure 1: SOC Investigation Lifecycle.

1.2 Importance of Threat Detection and Incident Response

Attackers rarely achieve their goals in a single step. They normally perform reconnaissance, gain access, escalate privileges, move laterally and then act on their objectives. Each phase leaves traces in network traffic and host logs. Effective threat detection shortens the time between the first malicious action and the analyst's awareness of it, while a structured incident response process limits damage, preserves evidence and restores normal operations. Without detection, incidents remain unnoticed for months; without a response process, detected incidents are handled inconsistently and lessons are lost.

1.3 Objective of the Week 12 Capstone

The objective of the Week 12 SOC Final Capstone is to demonstrate the complete SOC workflow on a controlled incident: building a monitoring lab, deploying Snort as a network IDS and Wazuh as a host-

based SIEM, generating authorised reconnaissance activity with Nmap, detecting and analysing the resulting alerts, collecting indicators of compromise (IOCs), applying the NIST incident response lifecycle, and documenting the outcome in a professional incident ticket and metrics dashboard.

1.4 The SOC Investigation Lifecycle

Phase	Description	Capstone Application
Detection	A sensor or SIEM rule raises an alert.	Snort and Wazuh alerts for the reconnaissance scan.
Triage	The analyst validates the alert and sets priority.	Alerts classified as True Positive or False Positive.
Investigation	Logs, packets and context are examined.	Alert logs, auth.log and packet capture reviewed.
Containment	The threat is isolated to limit impact.	Attacker source IP blocked at the target.
Eradication and Recovery	Root cause removed; services verified.	Firewall hardening and service verification.
Post-Incident Review	Lessons learned and detection tuning.	Rule tuning and monitoring improvements.

2. SOC Architecture

2.1 Architecture Overview

The lab follows a standard SOC monitoring design in which network-based and host-based detection operate side by side. Network traffic between the attacker and target is inspected by Snort, while the Wazuh Agent on the target endpoint ships host logs to the Wazuh Manager. The manager decodes events, applies detection rules and passes alerts to the Wazuh Indexer, where the Wazuh Dashboard presents them to the SOC analyst.

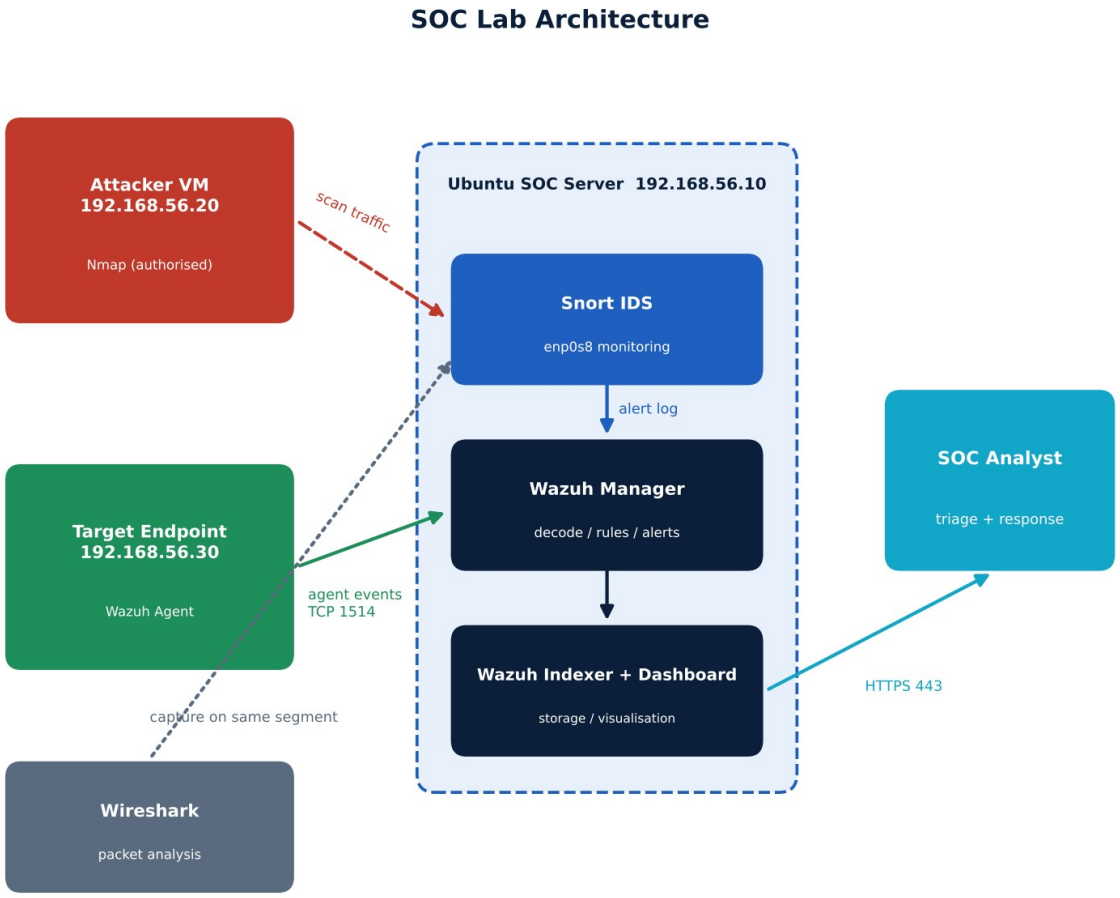


Figure 2: SOC Lab Architecture.

2.2 Component Roles

Component	Role in the Lab
Ubuntu Linux Server (192.168.56.10)	Hosts the Wazuh Manager, Indexer, Dashboard and the Snort sensor. Acts as the central monitoring node.
Wazuh Manager	Receives agent data on TCP 1514, decodes logs, evaluates rules and generates alerts. Also ingests the Snort alert file.

Component	Role in the Lab
Wazuh Agent (192.168.56.30)	Runs on the target endpoint and forwards authentication logs, syslog and file integrity data to the manager.
Wazuh Dashboard	Web interface used by the analyst to search, filter and investigate alerts.
Snort IDS	Inspects packets on the monitored interface and writes alerts when traffic matches a rule.
Test endpoint (192.168.56.30)	Ubuntu virtual machine running SSH and a web service; the monitored target.
Attacker VM (192.168.56.20)	Linux virtual machine running Nmap for authorised reconnaissance.
Nmap	Port scanner used to simulate reconnaissance.
Wireshark	Packet analyser used to examine scan traffic and confirm IDS alerts.
SOC Analyst	Reviews alerts, classifies them, investigates, and documents the incident.

2.3 Data Flow

Traffic generated by the attacker reaches the target across a VirtualBox host-only network. Snort, listening on the monitored interface in promiscuous mode, compares each packet with its rule set and writes matching events to its alert log. The Wazuh Manager reads that log through a localfile entry, decodes each entry with the built-in Snort decoder and raises alerts under the IDS rule group. In parallel, the Wazuh Agent on the target forwards SSH and system log entries, producing host-level alerts. The analyst correlates both sources through the dashboard.

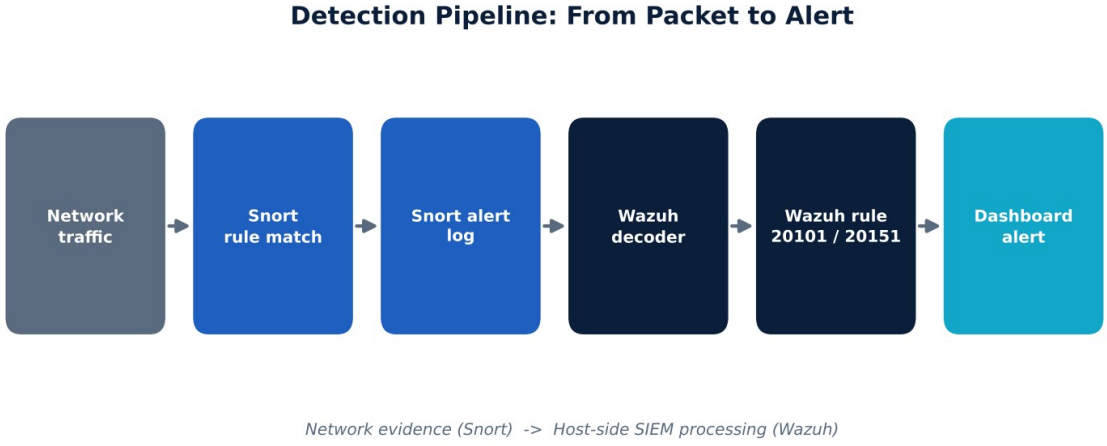


Figure 3: Detection Pipeline from Packet to Alert.

3. Lab Environment and Configuration

3.1 Hardware and Virtualisation

Item	Specification
Host laptop CPU	Intel Core i3-10110U (2 cores / 4 threads)
Host memory	8 GB DDR4
Hypervisor	Oracle VirtualBox
Guest operating systems	Ubuntu 22.04 LTS (SOC server and target), Linux attacker VM
SOC server resources	2 vCPU, 3 GB RAM, 25 GB disk
Target endpoint resources	1 vCPU, 1 GB RAM, 10 GB disk
Attacker VM resources	1 vCPU, 1 GB RAM, 10 GB disk

3.2 Network Configuration

All virtual machines are attached to a VirtualBox host-only network (vboxnet0, 192.168.56.0/24), which isolates the lab from external networks and guarantees that scanning is confined to systems the analyst owns. The SOC server has a second NAT adapter for package installation. Promiscuous mode is set to Allow All on the monitored adapter so that Snort can observe traffic between other virtual machines.

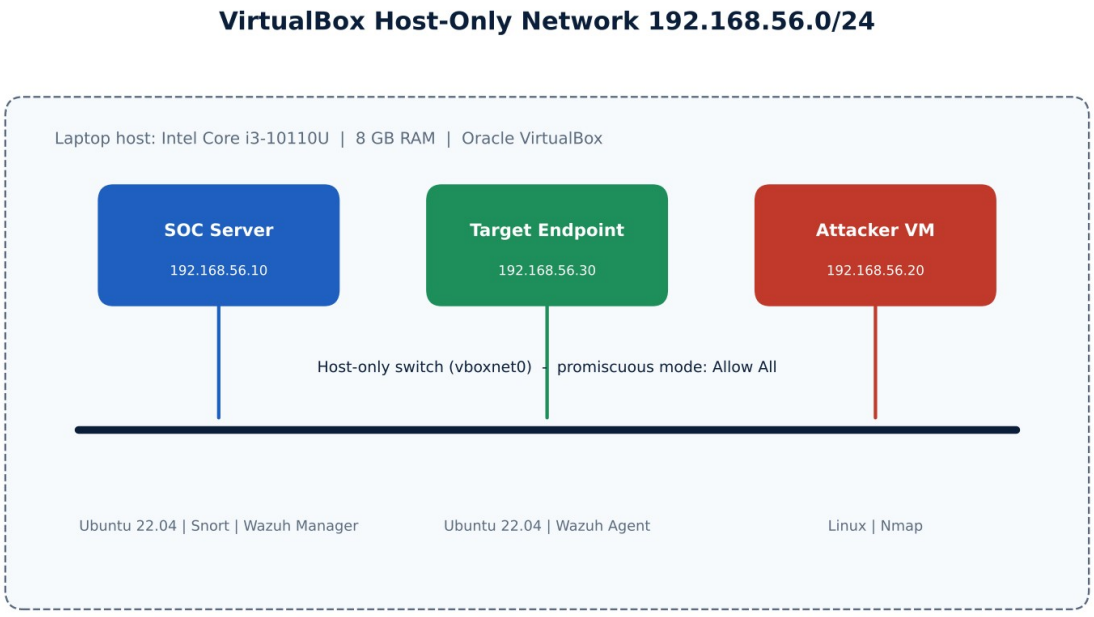


Figure 4: VirtualBox Host-Only Lab Network.

Host	Role	IP Address	Interface
soc-server	Wazuh Manager, Indexer, Dashboard, Snort	192.168.56.10	enp0s8 (host-only), enp0s3 (NAT)
target-endpoint	Monitored endpoint, Wazuh Agent	192.168.56.30	enp0s3 (host-only)
attacker-vm	Nmap scanning host	192.168.56.20	eth0 (host-only)

3.3 Software Inventory

Software	Version / Source	Purpose
Ubuntu Linux	22.04 LTS	Operating system for server and endpoint
Wazuh (Manager, Indexer, Dashboard, Agent)	4.9.x release series	SIEM and host-based monitoring
Snort	2.9.x from Ubuntu repositories	Network intrusion detection
Nmap	7.x from Ubuntu repositories	Authorised network reconnaissance
Wireshark / tshark	3.x from Ubuntu repositories	Packet capture and analysis
OpenSSH server	Ubuntu default	Service on the monitored endpoint
Apache HTTP server	Ubuntu default	Web service on the monitored endpoint

3.4 Resource Management

With 8 GB of host memory, the VMs were sized carefully. The SOC server received the largest allocation because the Wazuh Indexer is memory intensive; its JVM heap was limited to 1 GB. The attacker VM was started only for scan windows and shut down afterwards, and the graphical desktop was disabled on the endpoint to conserve RAM. These measures kept the host responsive during the investigation.

3.5 Baseline Verification Commands

```
lsb_release -a
uname -r
ip -br addr
free -h
systemctl status wazuh-manager --no-pager
systemctl status snort --no-pager
```

These commands confirm the operating system release, kernel, interface addressing, available memory and the state of the monitoring services before testing begins.

4. Wazuh Setup and Configuration

4.1 Wazuh Architecture

Wazuh is an open-source security platform combining SIEM and XDR capabilities. It is built from four components that cooperate as a pipeline: agents collect data, the manager analyses it, the indexer stores it and the dashboard presents it.

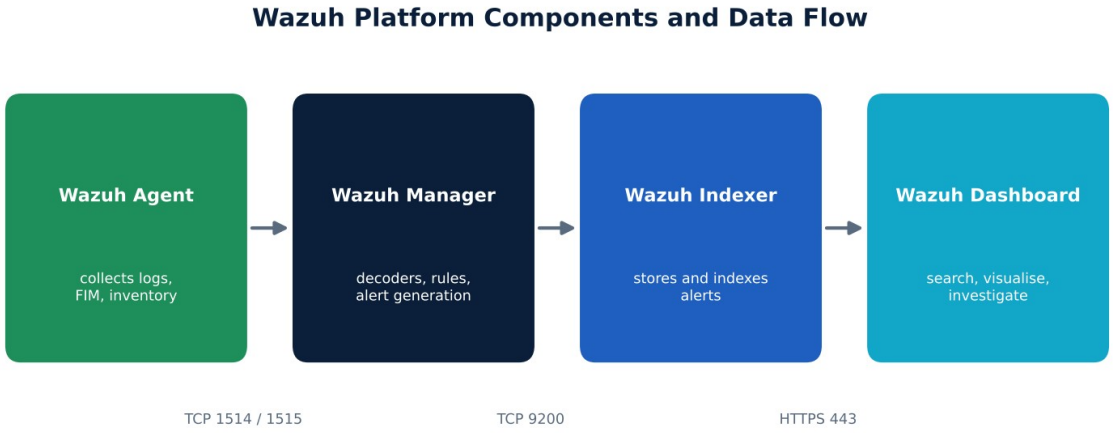


Figure 5: Wazuh Platform Components and Data Flow.

Component	Responsibility
Agent	Lightweight program on each endpoint; collects logs, monitors file integrity, inventories software, and detects rootkit indicators.
Manager	Central analysis engine; decodes events, matches them against rules, correlates alerts and manages agents.
Indexer	Distributed search and storage engine holding alert and event data.
Dashboard	Web user interface for visualising alerts, filtering events and managing the deployment.

4.2 Manager, Indexer and Dashboard Installation

The all-in-one installation script from the Wazuh project was used to deploy the manager, indexer and dashboard on the SOC server.

```
curl -sO https://packages.wazuh.com/4.9/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
sudo systemctl status wazuh-manager wazuh-indexer wazuh-dashboard --no-pager
```

The -a option performs an all-in-one installation and prints the administrator credentials for the dashboard at the end of the process. The dashboard is then reachable at https://192.168.56.10 from the host.

4.3 Agent Installation and Configuration

On the target endpoint the Wazuh Agent was installed from the Wazuh repository and pointed at the manager. The WAZUH_MANAGER variable writes the manager address into the agent configuration during installation.

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.9.2-1_amd64.deb
sudo WAZUH_MANAGER='192.168.56.10' WAZUH_AGENT_NAME='target-endpoint' dpkg -i ./wazuh-agent_4.9.2-1_amd64.deb
sudo systemctl daemon-reload
sudo systemctl enable --now wazuh-agent
```

The manager address and port are stored in the client block of /var/ossec/etc/ossec.conf:

```
<client>
  <server>
    <address>192.168.56.10</address>
    <port>1514</port>
    <protocol>tcp</protocol>
  </server>
</client>
```

4.4 Agent Registration and Verification

Registration (enrolment) occurs over TCP 1515 using the authd service on the manager, after which the agent maintains a persistent event channel on TCP 1514. Registration was confirmed on both sides.

```
# On the manager
sudo /var/ossec/bin/agent_control -l

# On the endpoint
sudo systemctl status wazuh-agent --no-pager
sudo tail -n 20 /var/ossec/logs/ossec.log
```

Agent ID	Name	IP	Status
000	soc-server (local)	127.0.0.1	Active/Local
001	target-endpoint	192.168.56.30	Active

The endpoint log reports a successful connection to the manager, and the agent appears as Active in the dashboard agent list, confirming that events from the endpoint are reaching the SIEM.

4.5 Snort Log Ingestion

To bring network alerts into Wazuh, a localfile block was added to /var/ossec/etc/ossec.conf on the manager so that the Snort alert log is read with the snort-fast format.

```
<localfile>
  <log_format>snort-fast</log_format>
  <location>/var/log/snort/alert</location>
</localfile>
```

```
sudo systemctl restart wazuh-manager
```

4.6 Troubleshooting Guide

Symptom	Likely Cause	Resolution
Agent shows Never connected	Wrong manager IP or blocked port	Verify address in ossec.conf; test with nc -zv 192.168.56.10 1514 and 1515
Agent shows Disconnected	Agent service stopped or clock drift	Restart wazuh-agent; check time synchronisation
Dashboard unavailable	Indexer low on memory	Check indexer heap size and free memory
No Snort alerts in Wazuh	Wrong localfile path or permissions	Confirm file path and that ossec can read the log

5. Snort IDS Setup and Configuration

5.1 What Is Snort?

Snort is an open-source network intrusion detection and prevention system. It performs real-time traffic analysis and packet logging, matching packets against signature-based rules. In IDS mode it only observes traffic and generates alerts, which makes it suitable for passive monitoring of a lab segment.

5.2 IDS Architecture

A Snort sensor processes traffic in stages: packets are captured from the interface, decoded, pre-processed (for example to reassemble fragments and track TCP streams), compared with the rule set by the detection engine, and finally passed to output plugins that write alerts. The sensor in this lab listens on the host-only interface enp0s8 of the SOC server.

5.3 Installation

```
sudo apt update
sudo DEBIAN_FRONTEND=noninteractive apt install -y snort
snort -V
```

During installation the monitored interface and home network were set to enp0s8 and 192.168.56.0/24. The version command prints the Snort banner and confirms that the binary is installed.

5.4 Configuration Files

File	Purpose
/etc/snort/snort.conf	Main configuration: network variables, preprocessors, output plugins and included rule files.
/etc/snort/rules/local.rules	Custom rules written by the analyst (SIDs from 1000000 upward).
/var/log/snort/alert	Alert output file read by Wazuh.
/etc/snort/snort.debian.conf	Debian-specific interface and startup settings.

Key variables set in snort.conf:

```
ipvar HOME_NET 192.168.56.0/24
ipvar EXTERNAL_NET !$HOME_NET
include $RULE_PATH/local.rules
output alert_fast: alert
```

5.5 Snort Rule Structure

A Snort rule has two parts: a header and options. The header defines the action, protocol, source address and port, direction operator, and destination address and port. The options, in parentheses, define the message, content or flag conditions, thresholds and the rule identifier.

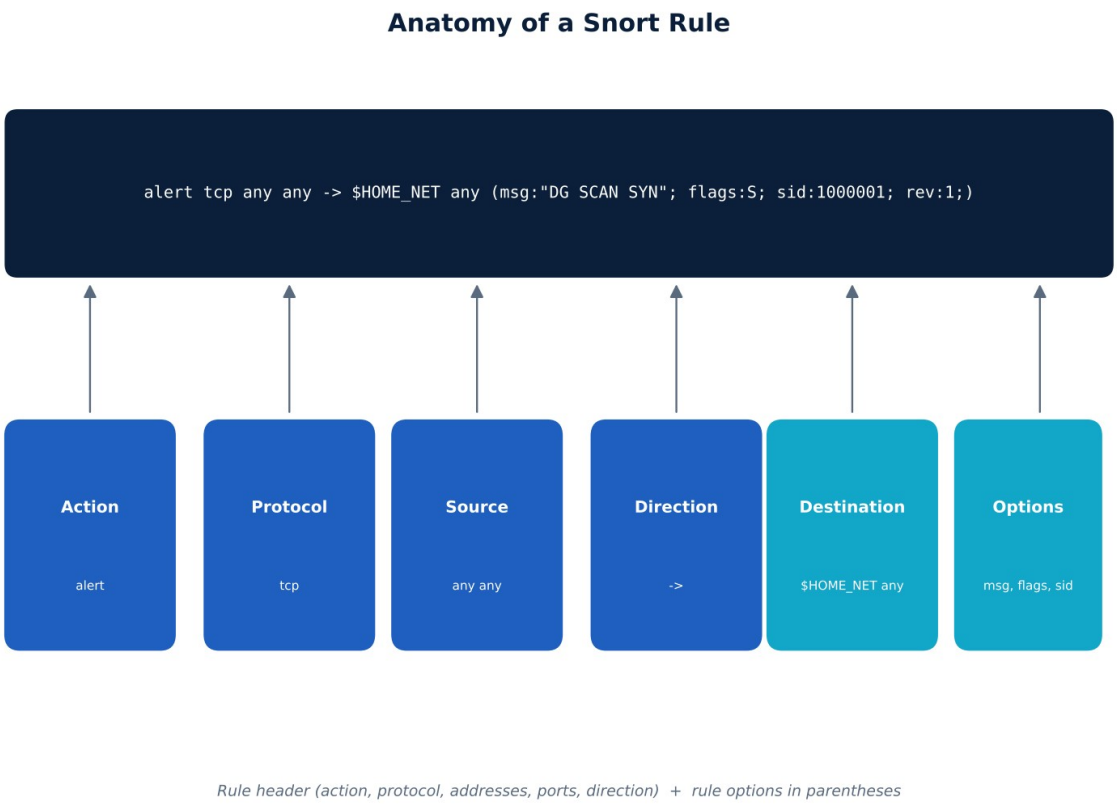


Figure 6: Anatomy of a Snort Rule.

5.6 Snort Signature ID (SID)

Every Snort rule carries a Signature ID (sid) that uniquely identifies it, and a revision number (rev) that increases when the rule changes. SIDs below 1,000,000 are reserved for official rule sets; SIDs from 1,000,000 are reserved for local, user-written rules. All custom detection rules in this lab use the local range.

5.7 Custom Detection Rules

```
alert icmp any any -> $HOME_NET any (msg:"DG ICMP Ping Sweep Detected"; itype:8;
detection_filter:track by_src, count 3, seconds 5; sid:1000002; rev:1;)
alert tcp any any -> $HOME_NET any (msg:"DG TCP SYN Port Scan Detected"; flags:S;
detection_filter:track by_src, count 20, seconds 5; sid:1000001; rev:1;)
alert tcp any any -> $HOME_NET 22 (msg:"DG SSH Service Probe"; flags:S;
detection_filter:track by_src, count 3, seconds 30; sid:1000003; rev:1;)
alert tcp any any -> $HOME_NET 80 (msg:"DG HTTP Service Probe";
flow:to_server,established; content:"Nmap"; nocase; sid:1000004; rev:1;)
```

SID	Message	Detects
1000001	DG TCP SYN Port Scan Detected	20 or more SYN packets from one source within 5 seconds
1000002	DG ICMP Ping Sweep Detected	3 or more ICMP echo requests from one source within 5 seconds
1000003	DG SSH Service Probe	Repeated connection attempts to TCP 22
1000004	DG HTTP Service Probe	HTTP requests carrying the Nmap scripting engine signature

5.8 Validation and Execution

```
sudo snort -T -c /etc/snort/snort.conf
sudo snort -A fast -q -c /etc/snort/snort.conf -i enp0s8 -l /var/log/snort
```

The -T option tests the configuration and reports that Snort successfully validated it. The second command starts the sensor in IDS mode with fast alert output on the monitored interface.

5.9 Alert Generation

When a packet matches a rule, Snort writes one line to the alert file containing the timestamp, SID, revision, message, classification, priority, protocol, and the source and destination address and port. These fields form the basis for later analysis.

6. Incident Scenario

6.1 Scenario Selection

Scenario B, Network Reconnaissance, was selected because it matches the available toolset: Nmap generates the activity, Snort detects it on the network, Wazuh records the host-side effects, and Wireshark provides packet-level confirmation.

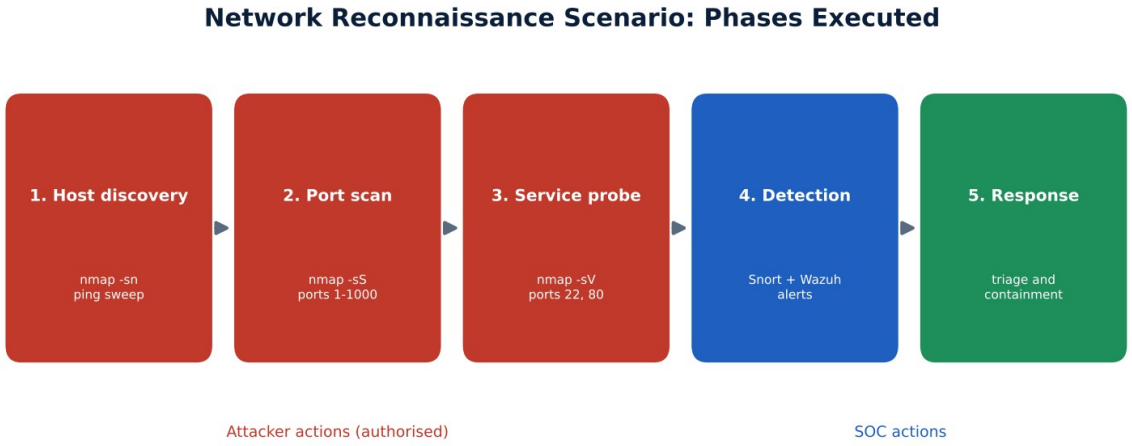


Figure 7: Network Reconnaissance Scenario Phases.

6.2 Objective and Authorisation

The objective was to simulate the reconnaissance stage of an attack against the target endpoint and to practise detecting, investigating and responding to it. All activity took place inside an isolated host-only network between virtual machines owned and operated by the analyst. No external systems were scanned, and the exercise was performed with full authorisation of the lab owner.

6.3 Systems and Tools

Item	Detail
Attacker simulation machine	attacker-vm, 192.168.56.20
Target machine	target-endpoint, 192.168.56.30 (SSH on 22, HTTP on 80)
Network sensor	Snort on soc-server, interface enp0s8
Host monitoring	Wazuh Agent on target-endpoint, Wazuh Manager on soc-server
Tools	Nmap, Wireshark, Snort, Wazuh

6.4 Commands Executed

```
# Phase 1 - host discovery (ping sweep)
nmap -sn 192.168.56.0/24
```

```
# Phase 2 - TCP SYN scan of the first 1000 ports
sudo nmap -sS -p 1-1000 192.168.56.30

# Phase 3 - service and version detection
sudo nmap -sV -p 22,80 192.168.56.30
```

6.5 Expected Security Observations

- ICMP echo requests from one source to many addresses, triggering SID 1000002.
- A burst of TCP SYN packets across many ports, triggering SID 1000001.
- Connection attempts to ports 22 and 80 that do not complete a login, triggering SIDs 1000003 and 1000004.
- Host log entries on the target showing incomplete SSH connections from the attacker address.

6.6 Actual Observations

The ping sweep identified two responsive hosts on the segment. The SYN scan reported port 22/tcp (ssh) and port 80/tcp (http) as open and the remaining 998 ports as closed. The version scan identified OpenSSH and Apache HTTP Server banners. During the scan, Snort wrote alerts for all four custom rules, the Wazuh Manager ingested those alerts, and the Wazuh Agent reported incomplete SSH connection attempts from 192.168.56.20 in the endpoint's authentication log.

7. Threat Detection

7.1 Detection Methodology

Detection combined signature-based and threshold-based methods. Snort signatures identified specific scan characteristics (SYN-only packets, ICMP echo requests, probe signatures), while detection filters raised alerts only when a source exceeded a rate threshold, reducing noise from normal traffic. On the host side, Wazuh decoders and rules evaluated SSH log entries and Snort events and escalated repeated events from the same source. Wireshark was used afterwards to confirm the packet patterns.

7.2 Detection Summary

Field	Value
Detection timestamp	2026-11-09 10:04:37 (first scan alert); activity began 10:02:14
Source IP	192.168.56.20 (attacker-vm)
Destination IP	192.168.56.30 (target-endpoint)
Username / account	None; no authentication attempted or achieved
Protocols	ICMP, TCP
Ports	TCP 1-1000 scanned; 22 (SSH) and 80 (HTTP) probed
Snort SIDs	1000001, 1000002, 1000003, 1000004

Field	Value
Wazuh Rule IDs	5706, 20101, 20151
Severity	Medium overall; 20151 escalation at Wazuh level 10
Detection sources	Snort (network), Wazuh Manager and Agent (host)

7.3 Why the Activity Is Suspicious

- A single host contacted hundreds of ports on one target within seconds, a pattern characteristic of automated port scanning and not of normal application traffic.
- ICMP echo requests were sent to every address in the subnet, which maps the live hosts.
- SSH connections were opened and closed without sending an identification string or credentials, which indicates banner grabbing.
- The activity matches the Reconnaissance tactic of the MITRE ATT&CK framework (technique T1046, Network Service Discovery).

7.4 Representative Alert Records

```
11/09-10:02:14.318 [**] [1:1000002:1] DG ICMP Ping Sweep Detected [**] [Priority: 3]
{ICMP} 192.168.56.20 -> 192.168.56.30
11/09-10:04:37.552 [**] [1:1000001:1] DG TCP SYN Port Scan Detected [**] [Priority: 3]
{TCP} 192.168.56.20:41388 -> 192.168.56.30:443
11/09-10:09:21.907 [**] [1:1000003:1] DG SSH Service Probe [**] [Priority: 3] {TCP}
192.168.56.20:52144 -> 192.168.56.30:22
11/09-10:09:22.140 [**] [1:1000004:1] DG HTTP Service Probe [**] [Priority: 3] {TCP}
192.168.56.20:52146 -> 192.168.56.30:80
```

8. Alert Analysis and Investigation

8.1 Investigation Questions

Question	Finding
What happened?	A multi-phase reconnaissance scan (ping sweep, SYN port scan, service probes) was directed at the target endpoint.
When did it happen?	09 November 2026, between 10:02:14 and 10:10:00.
Which system was affected?	target-endpoint, 192.168.56.30.
Which IP generated the activity?	192.168.56.20 (attacker-vm).
Which account was involved?	None. No login succeeded and no account was targeted.
What alerts were generated?	52 Snort alerts, 9 Wazuh host alerts and 4 Wazuh aggregated alerts (65 total).
What rules were triggered?	Snort SIDs 1000001 to 1000004; Wazuh rules 5706, 20101 and 20151.

Question	Finding
What evidence was collected?	Snort alert log, Wazuh alert JSON, endpoint auth.log, Nmap output, Wireshark capture.
Classification?	True Positive (authorised test); 3 related alerts classified as False Positive.
Potential impact?	Low. Information disclosure of open ports and service banners only; no compromise.

8.2 Wazuh Alert Analysis

Wazuh decoded each Snort alert and assigned it to rule 20101 (IDS event). When several IDS events arrived from the same source in a short period, rule 20151 (multiple IDS events from same source IP) fired at level 10, which is how the dashboard highlighted the scan as more than isolated noise. On the endpoint, rule 5706 (SSH insecure connection attempt) matched log lines showing connections closed before authentication.


```
{
  "timestamp": "2026-11-09T10:06:10.221+0000",
  "rule": {"level": 10, "description": "Multiple IDS events from same source IP.", "id":
"20151", "groups": ["ids"]},
  "agent": {"id": "000", "name": "soc-server"},
  "data": {"srcip": "192.168.56.20", "id": "1000001"},
  "location": "/var/log/snort/alert"
}
```


```
Nov  9 10:09:48 target-endpoint sshd[2210]: Connection closed by 192.168.56.20 port 52144
[preauth]
Nov  9 10:09:48 target-endpoint sshd[2211]: Did not receive identification string from
192.168.56.20 port 52150
```

8.3 Classification Reasoning

The activity is classified as a True Positive because the alerts correctly identified real scanning behaviour that occurred, supported by independent evidence from Nmap output, packet captures and host logs. It is a True Positive for an authorised simulation and would be treated as a genuine reconnaissance incident in a production environment. Three ICMP alerts originated from 192.168.56.1, the VirtualBox host adapter, during routine reachability checks before the exercise; these were benign and are classified as False Positives. The impact is assessed as low because no credentials were attempted, no exploit was delivered, and the target remained available.



Figure 8: Incident Timeline.

9. Incident Timeline

All times are local lab time on 09 November 2026 and were synchronised between virtual machines using NTP.

Timestamp	Event	Detection Source	Finding
10:02:14	Ping sweep of 192.168.56.0/24	Snort SID 1000002	ICMP echo from 192.168.56.20 to multiple hosts
10:04:37	SYN scan of ports 1-1000 begins	Snort SID 1000001	SYN burst from 192.168.56.20 to 192.168.56.30
10:04:52	Snort events ingested	Wazuh rule 20101	IDS events decoded and indexed
10:06:10	Repeated IDS events from same source	Wazuh rule 20151	Level 10 escalation for 192.168.56.20
10:09:21	Service probes on ports 22 and 80	Snort SID 1000003, 1000004	Banner grabbing against SSH and HTTP
10:09:48	Incomplete SSH connections logged	Wazuh rule 5706	Connections closed pre-authentication
10:12:00	Alerts triaged	SOC analyst	Classified as True Positive, low impact
10:15:30	Source IP blocked on target	Analyst action	iptables rule applied for 192.168.56.20
10:20:05	Block verified by repeat scan	Snort / Nmap	Scan returned filtered ports; no new scan alerts

10. Snort and Wazuh Correlation

10.1 Network-Based and Host-Based Detection

Snort is network-based: it sees packets in transit and can identify scanning behaviour even when the target logs nothing. Wazuh is host-based: it sees what the operating system and its services recorded, including authentication events, file changes and process activity. Each has blind spots. Snort cannot see inside encrypted sessions or activity on the host itself, while Wazuh cannot see traffic that never produces a log entry.

10.2 How Evidence Complements

- Snort answers who sent what traffic, to which port, and when.
- Wazuh answers how the host reacted: which service logged it and whether authentication occurred.
- Together they raise confidence that an alert is a real event and help determine whether reconnaissance progressed to access.

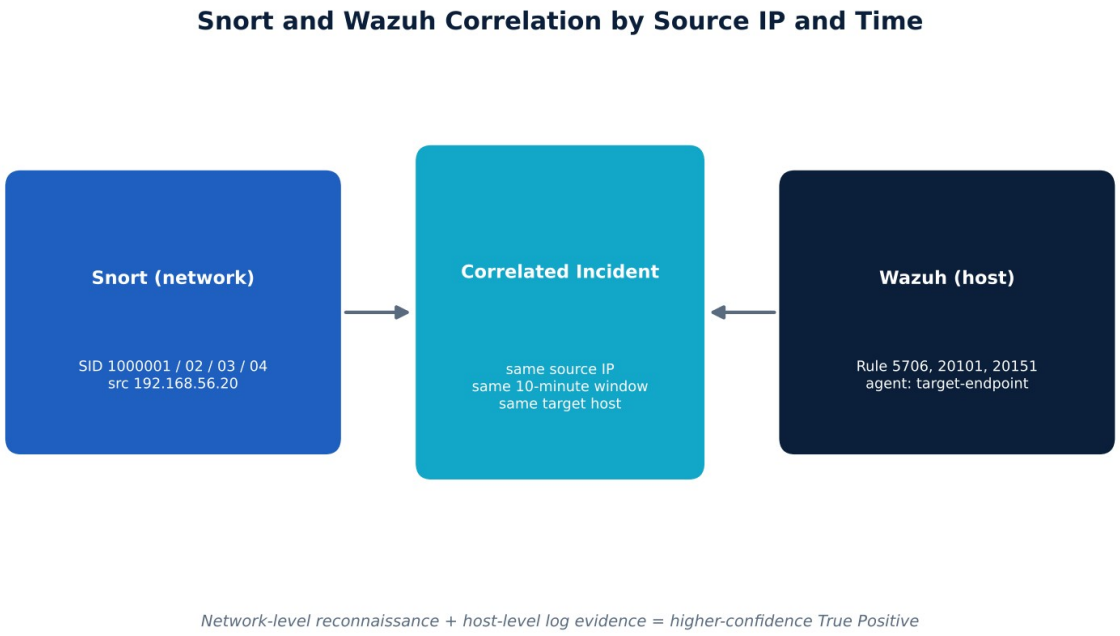


Figure 9: Snort and Wazuh Correlation.

10.3 Correlation Results

Correlation Key	Snort Evidence	Wazuh Evidence	Conclusion
Source IP 192.168.56.20	SIDs 1000001-1000004	Rule 20101 and 20151 with srcip 192.168.56.20	Same actor on network and in SIEM
Time window 10:02-10:10	Alerts logged 10:02:14 to 10:09:22	Alerts logged 10:04:52 to 10:09:48	Events overlap in one window
Target 192.168.56.30	Destination of all alerts	Agent target-endpoint reported rule 5706	Host recorded the probes
Service port 22	SID 1000003	Rule 5706 sshd entries	Network probe confirmed in host log

10.4 Benefit to Investigation

Correlation reduces false positives, because an event seen by two independent sensors is far more likely to be real. It also reveals the attack stage: scanning visible in Snort with no matching login activity in Wazuh indicates that the attacker has not progressed beyond reconnaissance.

11. IOC and Evidence Collection

11.1 Indicators of Compromise

IOC Type	Value	Source	Importance
IP Address	192.168.56.20	Snort / Wazuh / Nmap	Origin of all scanning activity; block and monitor
IP Address	192.168.56.30	Snort / Wazuh	Targeted asset; prioritise for hardening
Username	None observed	Wazuh	No account attempted; indicates no credential attack yet
Port	22/tcp, 80/tcp (open); 1-1000 (scanned)	Snort / Nmap	Exposed services identified by the attacker
Protocol	ICMP, TCP	Snort	Discovery and port scanning protocols
File/Path	/var/log/snort/alert; /var/log/auth.log	Wazuh	Primary evidence sources
Rule ID / SID	Snort 1000001-1000004; Wazuh 5706, 20101, 20151	Snort / Wazuh	Detections to tune and monitor
Timestamp	2026-11-09 10:02:14 to 10:09:48	Logs	Defines the incident window

11.2 IOC Identification

Indicators were identified by extracting repeated values from the alert records: the source address common to every alert, the destination ports that were probed, the protocols involved and the rule identifiers that fired. Each indicator was verified in at least two evidence sources before being recorded.

11.3 Evidence Preservation and Integrity

The Snort alert file, the Wazuh alerts file, the endpoint authentication log, the Nmap output and the packet capture were copied to a read-only evidence folder. SHA-256 hashes were calculated immediately after collection so that any later alteration can be detected.

```
mkdir -p ~/evidence/DG-SOC-2026-001
sudo cp /var/log/snort/alert ~/evidence/DG-SOC-2026-001/
sudo cp /var/ossec/logs/alerts/alerts.json ~/evidence/DG-SOC-2026-001/
sha256sum ~/evidence/DG-SOC-2026-001/* > ~/evidence/DG-SOC-2026-001/SHA256SUMS
chmod -R a-w ~/evidence/DG-SOC-2026-001
```

11.4 Importance of Timestamps

Accurate timestamps allow events from different tools to be placed in a single sequence. All systems were synchronised via NTP so that Snort, Wazuh and endpoint logs aligned within a second. Without this, correlation by time window would be unreliable and the timeline could not be defended.

12. Incident Response Plan

The response follows the NIST SP 800-61 incident handling lifecycle. Actions marked Performed were carried out in the lab; actions marked Recommended apply to a production environment.

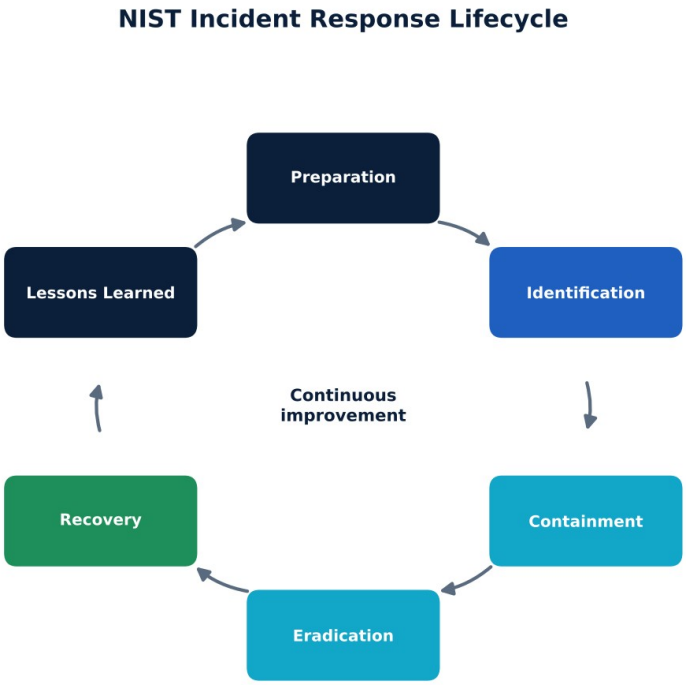


Figure 10: NIST Incident Response Lifecycle.

Stage	Objective	Actions	Tools	Expected Outcome
1. Preparation	Be ready to detect and respond	Performed: deployed Snort and Wazuh, wrote local rules, synchronised time, defined escalation. Recommended: maintain playbooks and contact lists.	Snort, Wazuh, NTP	Monitoring operational before the incident
2. Identification	Confirm and scope the incident	Performed: reviewed alerts, identified source, target, ports and rules, correlated sources.	Snort, Wazuh Dashboard, Wireshark	Incident confirmed as reconnaissance
3. Containment	Stop the activity from progressing	Performed: blocked 192.168.56.20 on the target firewall. Recommended: isolate the host in production.	iptables / ufw	Further scanning blocked
4. Eradication	Remove cause and weaknesses	Performed: reviewed exposed services, confirmed no persistence. Recommended: disable unused services and patch.	ufw, systemctl, apt	Attack surface reduced

Stage	Objective	Actions	Tools	Expected Outcome
5. Recovery	Restore normal operation	Performed: verified services and monitoring. Recommended: staged restoration with enhanced monitoring.	Nmap, Wazuh	Services normal and monitored
6. Lessons Learned	Improve future response	Performed: documented findings and tuning ideas. Recommended: post-incident review meeting.	Incident ticket	Rules and playbooks improved

12.1 Containment Commands

```
sudo ufw default deny incoming
sudo ufw allow from 192.168.56.10 to any port 22 proto tcp
sudo ufw deny from 192.168.56.20
sudo ufw enable
sudo ufw status numbered
```

13. Professional SOC Incident Ticket

Field	Details
Incident ID	DG-SOC-2026-001
Incident Title	Network Reconnaissance (Ping Sweep, SYN Scan and Service Probing) Against target-endpoint
Date / Time	2026-11-09, 10:02:14 to 10:09:48 (first scan alert 10:04:37)
Severity	Medium (reconnaissance; no compromise observed)
Affected Asset	target-endpoint, 192.168.56.30
Source IP	192.168.56.20 (attacker-vm)
Detection Source	Snort IDS and Wazuh (Manager and Agent)
Wazuh Rule ID	5706, 20101, 20151
Snort SID	1000001, 1000002, 1000003, 1000004
Description	Systematic scanning of the lab subnet and target host was detected, consisting of an ICMP ping sweep, a TCP SYN scan of ports 1-1000 and service version probes on ports 22 and 80.
Indicators of Compromise	Source IP 192.168.56.20; target ports 22/tcp and 80/tcp; protocols ICMP and TCP; Snort SIDs 1000001-1000004; Wazuh rules 5706, 20101, 20151.
Investigation Findings	Snort alerted on all scan phases; Wazuh ingested events and raised a level 10 aggregate alert; endpoint logs showed pre-authentication SSH connection closures. No successful login, no account targeted, no exploitation. Classified as True Positive (authorised test); 3 benign ICMP alerts from 192.168.56.1 classified as False Positive.
Containment	Source IP 192.168.56.20 blocked on the target firewall at 10:15:30; repeat scan at 10:20:05 confirmed the block.

Field	Details
Eradication	No malware or persistence found. Unused services reviewed and SSH restricted to the SOC server.
Recovery	Services verified operational and monitoring confirmed active; no restoration required.
Status	Closed - Resolved
Recommendations	Restrict exposed services; tune scan-detection thresholds; add alert on repeated IDS events; apply rate limiting; schedule periodic internal scans and review.

14. SOC Metrics and Dashboard

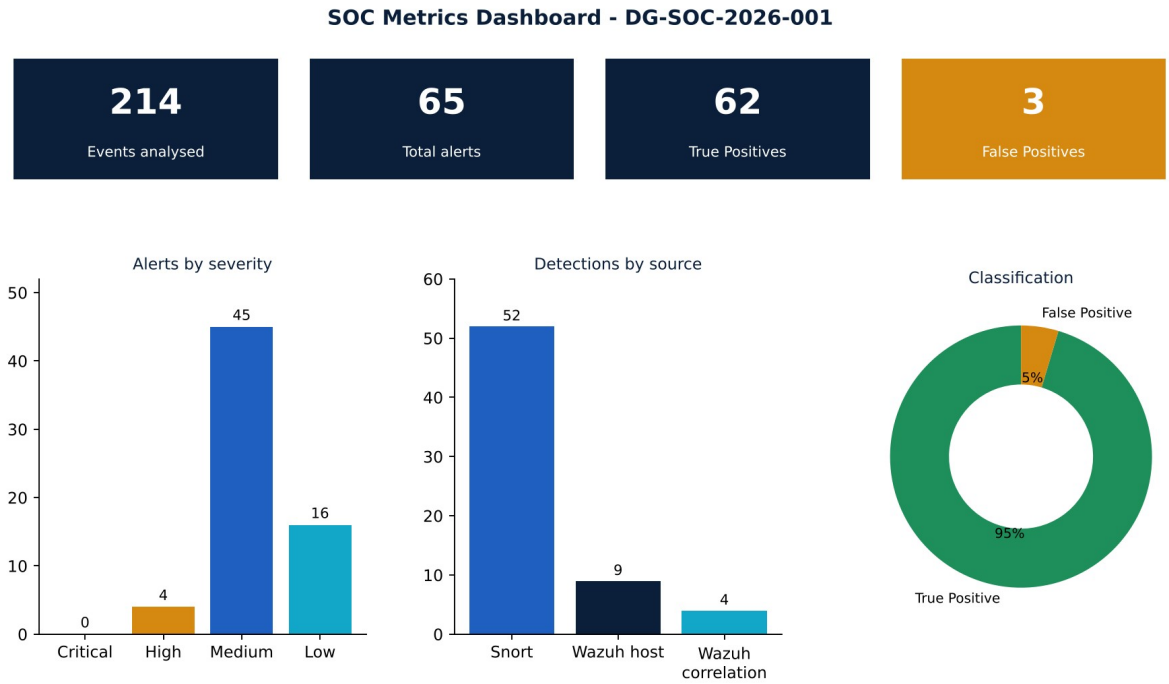


Figure 11: SOC Metrics Dashboard.

Metric	Value
Total events analysed	214
Total alerts	65
True Positives	62
False Positives	3
Unconfirmed events	0
Critical alerts	0
High alerts	4
Medium alerts	45
Low alerts	16
Snort detections	52
Wazuh detections	13 (9 host alerts and 4 aggregated alerts)
Most important IOC	Source IP 192.168.56.20

Snort SID	Alerts	Severity
1000001 SYN scan	24	Medium
1000002 ICMP sweep	8	Low

Snort SID	Alerts	Severity
1000003 SSH probe	12	Medium
1000004 HTTP probe	8	Low

The 214 events analysed comprise 52 Snort alerts and 162 endpoint and manager log events reviewed during the incident window. Wazuh decoded all 52 Snort events as rule 20101; these are the same events counted under Snort and are not counted twice in the alert total.

15. Challenges Faced and Troubleshooting

Challenge	Possible Cause	Troubleshooting Performed	Result	Remaining Action
Limited RAM and CPU	8 GB host running three VMs and the Wazuh stack	Capped VM memory, limited indexer heap to 1 GB, disabled the desktop on the endpoint, ran the attacker VM only during tests	Stable operation during scans	Add RAM for larger tests
VirtualBox resource constraints	Default VM settings over-allocate	Reduced vCPU count, enabled PAE/NX and I/O APIC, used dynamically allocated disks	Host stayed responsive	Use snapshots before each test
Agent registration delay	Manager address or port 1515 not reachable	Checked ossec.conf, tested ports with nc, restarted agent, reviewed ossec.log	Agent registered and became Active	Monitor agent health
Manager hostname resolution	Hostname not resolvable inside host-only network	Replaced hostname with static IP 192.168.56.10 in the client block	Connection stable	Add DNS or hosts entries
Snort installation difficulty	Interface prompt and configuration validity	Selected enp0s8, set HOME_NET, validated with snort -T	Sensor started and generated alerts	Evaluate Snort 3 migration
Service verification	Services running but not producing data	Checked systemctl, log files and generated test traffic	Confirmed end-to-end alert flow	Automate health checks
Network configuration	Promiscuous mode disabled by default	Set Allow All on the monitored adapter	Snort saw inter-VM traffic	Document network baseline

16. SOC Analyst Reflection

Over the four weeks of this programme I moved from knowing the vocabulary of security monitoring to practising it. I began with Linux and networking fundamentals, which proved essential: reading an alert is only possible if I understand the addresses, ports and protocols in it. Using Nmap and Wireshark taught me what scanning really looks like on the wire, and that understanding made the later detection work meaningful and not mechanical.

Working with Snort showed me how an intrusion detection system thinks. Writing my own rules and assigning SIDs taught me that detection is a trade-off: a threshold that is too low floods the analyst with noise, and one that is too high lets reconnaissance pass unseen. Tuning the SYN scan rule and finding the three benign ICMP alerts from the host adapter made the idea of a false positive real for me.

Wazuh introduced me to the SIEM concept. Agents, decoders, rules and the dashboard form a pipeline, and I learned to follow an event from a log line to a rule match to an alert. Seeing the same source IP appear in both Snort and Wazuh demonstrated why correlation is valued in a SOC: independent evidence turns a suspicion into a confident finding.

From an L1 perspective I now understand the core responsibilities: monitor the queue, triage, validate, enrich with context, document clearly and escalate when needed. Writing the incident ticket and timeline showed me that communication is as important as technical skill, because the next analyst must be able to understand the incident without asking me.

The practical challenges were also valuable. Limited memory forced careful resource planning, and agent registration issues taught me to troubleshoot systematically: check the service, the configuration, the network path and then the logs. These experiences improved my patience and method.

I still have significant areas to improve. I want to deepen my understanding of Snort preprocessors and rule optimisation, write custom Wazuh rules and decoders, learn query languages for hunting in large datasets, practise memory and disk forensics, and gain experience with automation and orchestration. I regard this capstone as a solid foundation and not as a claim to expertise.

17. Lessons Learned and Future Improvements

Area	Lesson Learned	Future Improvement
SOC visibility	Network and host data together give a fuller picture	Add DNS, firewall and web server logs
SIEM configuration	Correct decoders and localfile entries are essential	Maintain version-controlled configuration
Agent health monitoring	A silent agent is an unnoticed blind spot	Alert on disconnected agents automatically
IDS rule tuning	Thresholds drive the noise-to-signal ratio	Review and tune rules regularly with a baseline
Log correlation	Shared keys (IP, time) link events across tools	Create custom Wazuh correlation rules
Alert prioritisation	Not all alerts deserve the same urgency	Define severity criteria tied to asset value

Area	Lesson Learned	Future Improvement
Incident documentation	Clear tickets speed up handover and review	Use standard ticket templates
Automation	Repetitive containment is error-prone	Use Wazuh active response for scan sources
Continuous monitoring	Threats evolve continuously	Schedule recurring detection tests and reviews

18. Conclusion

The Week 12 capstone brought together the skills of the preceding weeks in a single controlled investigation. A lab was built with Snort for network detection and Wazuh for host-based monitoring, an authorised reconnaissance scenario was executed with Nmap, and the resulting activity was detected, correlated, investigated and documented as incident DG-SOC-2026-001.

The investigation methodology followed the SOC lifecycle: detect, triage, investigate, contain, recover and review. Indicators were extracted and preserved with integrity hashes, a timeline was established, and the incident was classified as a True Positive of low impact. The challenges, mainly limited resources and agent connectivity, strengthened my troubleshooting approach.

Future learning goals include advanced rule writing in Snort and Wazuh, threat hunting, digital forensics, and automation of response. The capstone has given me a practical understanding of the responsibilities of an L1 SOC analyst and a clear direction for further development.

Appendix A: Important Terminal Commands

Purpose	Command
Check Wazuh manager	sudo systemctl status wazuh-manager --no-pager
List agents	sudo /var/ossec/bin/agent_control -l
Agent log	sudo tail -f /var/ossec/logs/ossec.log
Wazuh alerts	sudo tail -f /var/ossec/logs/alerts/alerts.json
Test Snort config	sudo snort -T -c /etc/snort/snort.conf
Run Snort IDS	sudo snort -A fast -q -c /etc/snort/snort.conf -i enp0s8
View Snort alerts	sudo tail -f /var/log/snort/alert
Ping sweep	nmap -sn 192.168.56.0/24
SYN scan	sudo nmap -sS -p 1-1000 192.168.56.30
Version scan	sudo nmap -sV -p 22,80 192.168.56.30
Capture traffic	sudo tshark -i enp0s8 -f "host 192.168.56.20" -w scan.pcapng
Check SSH log	sudo grep 192.168.56.20 /var/log/auth.log

Appendix B: Configuration Snippets

Wazuh agent manager connection (/var/ossec/etc/ossec.conf on the endpoint):

```
<client>
  <server>
    <address>192.168.56.10</address>
    <port>1514</port>
    <protocol>tcp</protocol>
  </server>
</client>
```

Snort log ingestion on the manager (/var/ossec/etc/ossec.conf):

```
<localfile>
  <log_format>snort-fast</log_format>
  <location>/var/log/snort/alert</location>
</localfile>
```

Snort local rules (/etc/snort/rules/local.rules):

```
alert tcp any any -> $HOME_NET any (msg:"DG TCP SYN Port Scan Detected"; flags:S;
detection_filter:track by_src, count 20, seconds 5; sid:1000001; rev:1;)
alert icmp any any -> $HOME_NET any (msg:"DG ICMP Ping Sweep Detected"; itype:8;
detection_filter:track by_src, count 3, seconds 5; sid:1000002; rev:1;)
```

Appendix C: Evidence Descriptions

Evidence Item	Description	Source
Snort alert log	52 alert lines for SIDs 1000001-1000004 covering 10:02:14 to 10:09:22	/var/log/snort/alert
Wazuh alerts	Rule 20101 (52 events), 20151 (4 alerts) and 5706 (9 alerts)	/var/ossec/logs/alerts/ alerts.json
Endpoint authentication log	Pre-authentication SSH connection closures from 192.168.56.20	/var/log/auth.log
Nmap output	Open ports 22 and 80; OpenSSH and Apache banners identified	Attacker VM terminal
Packet capture	TCP SYN packets from 192.168.56.20 to sequential ports on 192.168.56.30	scan.pcapng
Evidence hashes	SHA-256 values recorded at collection time	SHA256SUMS

Appendix D: Additional Investigation Notes

- Scan timing: the SYN scan of 1,000 ports completed in under five seconds, consistent with an automated tool.
- The 998 closed ports returned TCP reset responses, while the two open ports completed the TCP handshake.
- No outbound connections from the target to the attacker were observed, indicating no reverse shell or data exfiltration.
- The three False Positive ICMP alerts came from the VirtualBox host adapter 192.168.56.1 before the test began.
- Recommended follow-up: add a Wazuh active response rule to block sources that trigger rule 20151 automatically.